

Projeto de Lógica Computacional 1 - 2026/1 Professor Flávio L. C. de Moura
Universidade de Brasília - UnB

Alunas: Sara Lima Gaspar - 22101721
Cecília Carvalho de Santana - 17008002
Giovana de Oliveira Rocha - 251033028

Tema: **Correção do Bubble Sort com Lógica de Primeira Ordem**

A aplicação da Lógica de Primeira Ordem (LPO) na verificação formal de software representa um passo fundamental para garantir a confiabilidade de sistemas. Este projeto tem como objetivo principal utilizar a LPO para resolver o problema computacional não trivial da verificação formal de correção de algoritmos. Para isso, o presente trabalho detalha a prova formal de correção do clássico algoritmo de ordenação por bolha, conhecido como Bubble Sort.

Todo o desenvolvimento lógico e matemático foi mecanizado no assistente de provas Rocq. Essa ferramenta fundamenta-se no sistema de Dedução Natural, o que a torna um ambiente ideal para conectar as provas em papel construídas ao longo do curso de Lógica Computacional com a prática computacional. Além de permitir a verificação rigorosa, o assistente viabiliza a extração de código certificado para linguagens funcionais. Embora o Bubble Sort seja amplamente reconhecido por não ser o algoritmo de ordenação mais eficiente do ponto de vista de complexidade de tempo de execução, a sua escolha como objeto de estudo lógico e estrutural é altamente justificada pelo seu imenso valor pedagógico. A construção do pensamento algorítmico exige blocos de fundação claros. A intuição por trás das bolhas, que consiste em percorrer a lista sucessivas vezes comparando e trocando elementos adjacentes, é imediata. Essa simplicidade o torna o algoritmo perfeito para introduzir a ponte entre a intuição de um programador e a exigência de uma prova matemática.

O primeiro passo na formalização do algoritmo Bubble Sort é isolar o comportamento de uma única passagem pela estrutura de dados. Para isso, definimos a função auxiliar bubble, que recebe uma lista de naturais e a percorre comparando elementos consecutivos. Chamamos esse processo de borbulhamento. Do ponto de vista pedagógico e do estudo de estruturas de dados, essa função é um excelente laboratório para entender a natureza das chamadas recursivas. A lógica de implementação revela um detalhe importante: esta função não é estruturalmente recursiva porque a lista $(x::l)$ gerada no ramo **else** não é uma sublista direta da lista original $(x::y::l)$. Em sistemas de Dedução Natural e lógica construtiva, como o utilizado pelo assistente de provas Rocq, é exigido que todas as funções sejam totais, ou seja, que a terminação seja garantida. Neste caso, utilizamos o comando **Function** para construir esta função e precisamos fornecer uma medida explícita que decresce em cada chamada recursiva, além de provar que esta medida efetivamente decresce. A medida escolhida foi o tamanho da lista - **measure length l**. Tal requisito de ordem formal impõe a necessidade de apresentar uma justificativa matemática rigorosa da prova de que o procedimento de ordenação não resultará em um ciclo infinito de execução, fortalecendo a compreensão sobre a estabilidade do código.

A função principal, o algoritmo **Bubble Sort** propriamente dito, é dada pela função **bs**. Diferente da função auxiliar de borbulhamento, a função **bs** é estruturalmente recursiva.

Isso significa que ela opera consumindo diretamente a estrutura de dados: a cada chamada recursiva, a função recebe uma sublista estritamente menor, a cauda da lista original. Por possuir essa garantia natural de terminação, o assistente de provas permite que ela seja definida utilizando o comando **Fixpoint**, dispensando a necessidade de uma prova explícita de decréscimo de medida. Do ponto de vista didático, apresentar a função **bubble**, definida via **Function** com medida, lado a lado com a função **bs** oferece uma oportunidade valiosa para explorar as diferenças sintáticas e semânticas da recursão no desenvolvimento formal de software. O algoritmo **bs** opera de maneira muito elegante: ele ordena recursivamente o restante da lista e, em seguida, utiliza a função **bubble** para posicionar o elemento atual na posição correta dentro dessa sublista já processada.

Para provarmos que o algoritmo **bs** funciona, é necessário, antes de tudo, definir o comportamento matemático das suas peças fundamentais. Ao aplicar a função **bubble** a uma lista qualquer não necessariamente vai retornar uma lista ordenada em uma única passagem. No entanto, precisamos garantir a estabilidade do algoritmo. O lema **bubble_sorted** expressa uma propriedade fundamental: se uma lista já está ordenada, a aplicação da função de borbulhamento não altera a sua estrutura. Formalmente, o lema é declarado como: **forall l, Sorted l => bubble l = l**.

Para demonstrar essa propriedade, a prova mecanizada utiliza o **Princípio da Indução Estrutural** sobre a lista **l**. Como as listas são estruturas de dados definidas recursivamente, uma lista ou é vazia, ou é um elemento anexado a outra lista, a indução é a ferramenta lógica natural para varrer todos os casos possíveis.

No caso base avaliamos o comportamento da função para a lista vazia. Como o borbulhamento de uma lista vazia é trivialmente a própria lista vazia, a propriedade se mantém. Enquanto no passo indutivo assumimos como hipótese de indução que a propriedade é válida para o restante da lista e analisamos a inserção de um novo elemento no topo. Através da quebra de casos, mostramos que, como a premissa garante que a lista inteira está ordenada, a comparação lógica $x \leq y$ sempre será verdadeira para os primeiros elementos. Consequentemente, o algoritmo reconstrói a lista exatamente na mesma ordem, satisfazendo a igualdade matemática $l = l$.

Para evitar uma anomalia lógica, é estritamente necessário provar uma segunda propriedade: a lista de saída deve ser uma permutação exata da lista de entrada. Ou seja, a função não pode inserir novos valores, nem descartar os originais. Para formalizar essa conservação, o projeto utiliza as definições indutivas de **Permutation**.

A primeira garantia ocorre no nível da função auxiliar. O lema **bubble_perm**, formalizado por **forall l, Permutation l** assegura que o borbulhar uma lista apenas reposiciona os seus elementos. Na lógica construtiva, provar isso significa mapear as operações da função diretamente para a regra de inferência **perm_swap**, que atesta que trocar elementos adjacentes gera uma permutação válida.

Com a função auxiliar estabilizada, elevamos a propriedade de conservação para o algoritmo completo através do lema **bs_permuta**. A prova é construída via **Indução Estrutural**. Assumindo que a chamada recursiva para a cauda da lista preserva os elementos originais, o algoritmo simplesmente insere o elemento atual no topo e aplica o

bubble. Utilizando a regra de transitividade, concluímos logicamente que se a sublista é uma permutação, e o borbulhamento gera uma permutação, a saída final do Bubble Sort preserva integralmente a identidade da lista original.

Para consolidar a verificação do algoritmo Bubble Sort, o Teorema `bs_correto` une as duas propriedades fundamentais exploradas: a ordenação e a conservação dos dados. Na verificação formal de software, definir que um algoritmo de ordenação é correto significa provar matematicamente a conjunção exata dessas duas garantias. Se o algoritmo satisfizesse apenas a propriedade de ordenação (**Sorted**), ele poderia, por exemplo, apagar elementos ou retornar uma lista vazia para qualquer entrada e, ainda assim, a afirmação de que a saída está ordenada seria verdadeira. Por outro lado, se garantisse apenas a permutação (**Permutation**), o algoritmo poderia simplesmente devolver os dados embaralhados. Ao demonstrar formalmente através da **Lógica de Primeira Ordem** que a saída de **bs** é, simultaneamente, ordenada e uma permutação exata da entrada, o teorema estabelece a prova definitiva de que o algoritmo cumpre o seu propósito lógico e computacional de maneira íntegra.